BÀI TẬP & BÀI THỰC HÀNH

NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 8: Bế tắc (Deadlocks)

5 bài tập tự luận + 3 bài thực hành lập trình/Linux

PHẦN 1: BÀI TẬP

Bài tập 1

Xét một hệ thống gồm 4 luồng T0, T1, T2, T3 và 3 loại tài nguyên A, B, C. Tại thời điểm hiện tại, trạng thái cấp phát tài nguyên như sau:

Luồng	Allocation (A B C)	Max (A B C)
T0	1 0 2	4 2 2
T1	2 1 0	3 2 1
T2	1 2 1	3 3 3
T3	0 1 3	2 2 3

Available (A B C) = 2 1 1

a) Tính ma trận Need cho từng luồng (Need = Max − Allocation).

b) Áp dụng giải thuật an toàn (safety algorithm) đã học, hãy trình bày từng bước lặp (Work, Finish) và kết luận hệ thống hiện có đang ở trạng thái an toàn hay không. Nếu an toàn, hãy chỉ ra một dãy an toàn <T?, T?, T?, T?>.

Bài tập 2

Sử dụng lại đúng trạng thái gốc ở Bài tập 1 (Allocation, Need, Available như trên; không tích lũy kết quả giữa các câu). Xét độc lập 3 yêu cầu sau, mỗi yêu cầu áp dụng giải thuật yêu cầu tài nguyên cho luồng Pi (resource-request algorithm) trên trạng thái gốc:

a) T2 gửi yêu cầu Request2 = (1, 0, 1)

b) T0 gửi yêu cầu Request0 = (4, 0, 0)

c) T3 gửi yêu cầu Request3 = (1, 1, 0)

Với mỗi yêu cầu, hãy: (i) kiểm tra tính hợp lệ (Request ≤ Need?); (ii) kiểm tra tài nguyên khả dụng (Request ≤ Available?); (iii) nếu qua được hai bước trên, giả định cấp phát tạm thời rồi chạy lại giải thuật an toàn để xác định trạng thái mới có an toàn không; (iv) kết luận yêu cầu được CHẤP THUẬN hay bị TỪ CHỐI (phải chờ), nêu rõ lý do cho từng trường hợp.

Bài tập 3

a) Cho một đồ thị cấp phát tài nguyên gồm 4 luồng T1, T2, T3, T4 và 4 loại tài nguyên R1, R2, R3, R4, **mỗi loại chỉ có đúng 1 thực thể**, với các cạnh sau:

- T1 đang chờ R1 (cạnh yêu cầu T1 → R1)
- R1 đang được cấp cho T2 (cạnh cấp phát R1 → T2)
- T2 đang chờ R2 (T2 → R2)
- R2 đang được cấp cho T3 (R2 → T3)
- T3 đang chờ R3 (T3 → R3)
- R3 đang được cấp cho T4 (R3 → T4)
- T4 đang chờ R4 (T4 → R4)
- R4 đang được cấp cho T1 (R4 → T1)

Hãy xác định trong đồ thị có chu trình hay không (liệt kê chu trình nếu có), và kết luận hệ thống có đang bế tắc không. Giải thích dựa trên số thực thể của mỗi loại tài nguyên.

b) Cho một tình huống khác: R1 có **2 thực thể**. T1 đang giữ 1 thực thể của R1 và đang chờ R2 (T1 → R2). T2 đang giữ R2 và đang chờ R1 (T2 → R1). T3 đang giữ thực thể còn lại của R1 nhưng **không yêu cầu thêm tài nguyên nào cả** (T3 sẽ hoàn thành công việc và giải phóng tài nguyên trong thời gian hữu hạn). Hãy vẽ (mô tả bằng lời) đồ thị này, chỉ ra chu trình (nếu có), và kết luận hệ thống có chắc chắn bế tắc hay không. Giải thích tại sao kết luận ở câu b) có thể khác với câu a) dù cả hai đồ thị đều có chu trình.

Bài tập 4

Xét hệ thống gồm 4 luồng T0, T1, T2, T3 và 3 loại tài nguyên A, B, C (mỗi loại A, B, C hiện chỉ có đúng 1 thực thể còn khả dụng như Available bên dưới). Trạng thái tại một thời điểm:

Luồng	Allocation (A B C)	Request (A B C)
T0	1 0 0	0 0 0
T1	1 0 0	0 0 0
T2	0 1 0	0 0 1
T3	0 0 1	0 1 0

Available (A B C) = 1 0 0

Áp dụng giải thuật phát hiện bế tắc (deadlock detection algorithm) cho trường hợp nhiều thực thể tài nguyên: trình bày các bước lặp (Work, Finish) và cho biết luồng nào (nếu có)

đang bị bế tắc. Giải thích bản chất của tình huống bế tắc này (tài nguyên nào, giữa các luồng nào, theo kiểu chờ đợi vòng tròn như thế nào).

Bài tập 5

Cho đoạn mã giả sử dụng hai mutex `mutex_A` (số hiệu 3) và `mutex_B` (số hiệu 9):

```
Thread 1:                       Thread 2:

lock(mutex_B)                   lock(mutex_A)

lock(mutex_A)                   lock(mutex_B)

... (dùng tài nguyên) ...        ... (dùng tài nguyên) ...

unlock(mutex_A)                  unlock(mutex_B)

unlock(mutex_B)                  unlock(mutex_A)
```

a) Đối chiếu với bốn điều kiện cần của bế tắc (loại trừ tương hỗ, giữ và chờ, không thu hồi, chờ đợi vòng tròn), hãy phân tích xem đoạn mã trên có khả năng thỏa mãn cả bốn điều kiện hay không — giải thích rõ từng điều kiện được thể hiện ở đâu trong đoạn mã.

b) Hãy mô tả một kịch bản thực thi cụ thể theo mốc thời gian (t = 0, 1, 2, 3, …) của hai luồng khiến chương trình rơi vào bế tắc thật sự.

c) Áp dụng kỹ thuật phá vỡ điều kiện chờ đợi vòng tròn bằng cách áp đặt thứ tự toàn phần lên các mutex theo số hiệu tăng dần, hãy viết lại đoạn mã giả cho Thread 2 (hoặc Thread 1) sao cho không còn khả năng bế tắc, và giải thích ngắn gọn vì sao cách sửa này loại bỏ được bế tắc.

PHẦN 2: BÀI THỰC HÀNH

Bài thực hành 1: Cài đặt giải thuật Banker's Algorithm (kiểm tra trạng thái an toàn)

Mục tiêu: Hiểu và cài đặt được giải thuật an toàn (safety algorithm) dùng trong tránh bế tắc (deadlock avoidance) bằng ngôn ngữ C; củng cố cách tính ma trận Need và cách duyệt Work/Finish.

Yêu cầu: Môi trường Linux, trình biên dịch gcc hỗ trợ C11. Không cần thư viện ngoài chuẩn (`stdio.h`, `stdbool.h`).

Đề bài chi tiết: Viết chương trình C cài đặt giải thuật an toàn của Banker's Algorithm với dữ liệu đầu vào là ma trận Allocation, ma trận Max và vector Available (có thể khai báo cứng trong mã nguồn, dùng đúng số liệu của Bài tập 1). Chương trình phải: (1) tự tính ma trận Need = Max − Allocation và in ra; (2) chạy giải thuật an toàn; (3) in ra kết luận hệ thống an toàn hay không, và nếu an toàn thì in ra một dãy an toàn tìm được.

Các bước thực hiện:

1. Tạo file `banker.c`, khai báo các mảng `allocation[N][M]`, `max_demand[N][M]`, `available[M]` với N=4, M=3, dùng đúng số liệu Bài tập 1.
2. Viết hàm `compute_need()` tính ma trận Need.
3. Viết hàm `is_safe()` cài đặt đúng 4 bước của giải thuật an toàn đã học (Work, Finish, tìm i, cập nhật Work, lặp lại), lưu lại dãy an toàn tìm được.
4. Biên dịch: `gcc -Wall -pthread -o banker banker.c`
5. Chạy thử: `./banker` và đối chiếu kết quả với lời giải Bài tập 1 (dãy an toàn tìm được có thể khác thứ tự nhưng vẫn phải hợp lệ).
6. Thử đổi `available[M]` thành `{0,0,0}` rồi biên dịch/chạy lại để quan sát trường hợp hệ thống KHÔNG an toàn.

Bài thực hành 2: Minh họa và khắc phục deadlock thực sự giữa hai pthread

Mục tiêu: Quan sát trực tiếp một bế tắc thật (chương trình treo vĩnh viễn) do khóa 2 mutex theo thứ tự ngược nhau giữa 2 luồng POSIX thread, sau đó khắc phục bằng kỹ thuật thứ tự khóa cố định.

Yêu cầu: Linux, gcc (C11), thư viện `pthread.h`, `unistd.h`; biên dịch với cờ `-pthread`. Dùng thêm lệnh `timeout` của shell để giới hạn thời gian chạy khi kiểm chứng chương trình bị treo.

Đề bài chi tiết: Viết chương trình `deadlock.c` tạo 2 luồng: Thread 1 khóa `mutex_B` rồi khóa `mutex_A` (có `sleep(1)` xen giữa để đảm bảo tình huống race xảy ra); Thread 2 khóa `mutex_A` rồi khóa `mutex_B` (cũng có `sleep(1)` xen giữa). Chạy thử để xác nhận chương trình bị treo (không bao giờ in được dòng kết thúc). Sau đó viết phiên bản `fixed.c` sửa lại thứ tự khóa của Thread 1 giống Thread 2 (đều khóa mutex_A trước, mutex_B sau) để chương trình luôn kết thúc bình thường.

Các bước thực hiện:

1. Tạo file `deadlock.c` với 2 hàm luồng như mô tả (dùng `pthread_mutex_lock`, `sleep(1)`, `pthread_mutex_unlock`), có in log ra màn hình ở mỗi bước khóa/mở khóa.
2. Biên dịch: `gcc -Wall -pthread -o deadlock deadlock.c`
3. Chạy có giới hạn thời gian để không bị treo terminal: `timeout 5 ./deadlock; echo "exit code: $?"` — quan sát chương trình dừng in log giữa chừng và mã thoát trả về là 124 (do bị `timeout` giết vì treo).
4. Tạo file `fixed.c` là bản sao của `deadlock.c` nhưng sửa lại Thread 1 để khóa `mutex_A` trước rồi mới `mutex_B` (cùng thứ tự với Thread 2).
5. Biên dịch: `gcc -Wall -pthread -o fixed fixed.c`
6. Chạy nhiều lần liên tiếp: `for i in 1 2 3 4 5; do ./fixed; done` — quan sát chương trình luôn in đủ log và kết thúc bình thường (không cần `timeout`), chứng tỏ bế tắc đã được loại bỏ.

Bài thực hành 3: Cài đặt giải thuật phát hiện bế tắc (Deadlock Detection Algorithm)

Mục tiêu: Cài đặt được giải thuật phát hiện bế tắc cho trường hợp tài nguyên có nhiều thực thể (Available, Allocation, Request), xác định chính xác tập luồng bị bế tắc.

Yêu cầu: Linux, gcc (C11). Không cần thư viện ngoài chuẩn.

Đề bài chi tiết: Viết chương trình C dùng đúng số liệu ở Bài tập 4 (Allocation, Request, Available của 4 luồng T0–T3, 3 loại tài nguyên A, B, C). Chương trình phải in ra từng luồng khi nó "hoàn thành" trong quá trình chạy giải thuật, và cuối cùng in danh sách các luồng bị bế tắc (nếu có) hoặc thông báo hệ thống không bế tắc.

Các bước thực hiện:

1. Tạo file `detect.c`, khai báo `allocation[N][M]`, `request[N][M]`, `available[M]` theo đúng số liệu Bài tập 4.
2. Khởi tạo Work = Available; Finish[i] = true nếu tổng Allocation của luồng i bằng 0, ngược lại false.

3. Cài đặt vòng lặp tìm i thỏa Finish[i]==false và Request[i] ≤ Work; nếu tìm được thì cập nhật Work += Allocation[i], đặt Finish[i]=true, in log; lặp lại cho đến khi không còn tiến triển được nữa.

4. Sau vòng lặp, duyệt mảng Finish: in ra các luồng có Finish==false là luồng bị bế tắc.

5. Biên dịch: `gcc -Wall -pthread -o detect detect.c`

6. Chạy `./detect` và đối chiếu kết quả in ra (T2, T3 bị bế tắc) với lời giải Bài tập 4.

7. Thử sửa `request[2]` (Request của T2) thành `{0,0,0}` (không còn yêu cầu gì thêm), biên dịch lại và chạy để quan sát trường hợp hệ thống không còn bế tắc.

========== END ==========